

Performance Analysis of Binary Heap, Fibonacci Heap, and Hollow Heap

A Comparative Case Study Using Dijkstra's Algorithm

Project: Heaps & Shortest Path Algorithm Analysis

Course: Data Structures and Algorithms

Date: November 2024

1. Executive Summary

This report presents a comprehensive performance analysis of three advanced heap data structures—Binary Heap, Fibonacci Heap, and Hollow Heap—when integrated as priority queues within Dijkstra's shortest path algorithm. The study evaluates both theoretical complexity and empirical performance across multiple graph datasets, ranging from small synthetic graphs (301 vertices) to large real-world road networks (43,620 vertices).

Key Findings

- 1 Binary Heap** demonstrated superior overall performance in real-world graph scenarios, achieving a total runtime of 199.06ms on the Hong Kong road network dataset (43,620 vertices, 91,542 edges).
- 2 Fibonacci Heap** showed competitive performance with a 2.6× slower runtime (527.06ms) but excelled in decrease-key operations (0.03μs vs. 0.04μs for Binary Heap).
- 3 Hollow Heap** significantly underperformed with an 8.8× slower runtime (1754.93ms), despite its theoretical advantages in amortized complexity.

- 4 Binary Heap maintained consistent $O(\log n)$ performance with minimal structural overhead (height: 8, single tree structure).

Performance Summary - Hong Kong Road Network (43,620 vertices, 91,542 edges)

Heap Type	Insert Time (μs)	Extract-Min Time (μs)	Decrease-Key Time (μs)	Total Runtime (ms)	Height	#Trees
Binary Heap Best	0.06	0.09	0.04	199.06	8	1
Fibonacci Heap Good	0.06	0.35	0.03	527.06	9	7
Hollow Heap Poor	0.06	0.74	0.05	1754.93	1	732

Conclusion: The experimental results demonstrate a significant gap between theoretical complexity and practical performance. While Fibonacci and Hollow heaps offer superior amortized time bounds for decrease-key operations, Binary Heap's simpler structure, better cache locality, and lower constant-factor overhead result in faster execution times for typical shortest path workloads.

2. Theoretical Overview

This section provides a comprehensive analysis of the three heap data structures, examining their fundamental properties, theoretical complexity bounds, and structural

characteristics.

Binary Heap

A complete binary tree that satisfies the heap property, typically implemented using an array representation.

Structural Properties:

- **Storage:** Array-based representation
- **Shape:** Complete binary tree
- **Height:** $\lceil \log_2(n) \rceil$ for n elements
- **Memory Overhead:** Minimal (no explicit pointers)

Time Complexity:

- **Insert:** $O(\log n)$
- **Extract-Min:** $O(\log n)$
- **Decrease-Key:** $O(\log n)$
- **Find-Min:** $O(1)$

Fibonacci Heap

A collection of heap-ordered trees organized as a circular doubly-linked list with lazy consolidation.

Structural Properties:

- **Storage:** Pointer-based with circular doubly-linked root list
- **Shape:** Multiple heap-ordered trees
- **Marking:** Nodes marked after losing a child
- **Consolidation:** Performed lazily during extract-min

Time Complexity (Amortized):

- **Insert:** $O(1)$
- **Extract-Min:** $O(\log n)$
- **Decrease-Key:** $O(1)$
- **Find-Min:** $O(1)$

Hollow Heap

A relatively recent data structure that uses "hollow" nodes to simplify Fibonacci Heap operations while maintaining asymptotic bounds.

Structural Properties:

- **Storage:** Pointer-based with ranked nodes
- **Hollow Nodes:** Marked as hollow when deleted
- **Consolidation:** Performed during extract-min
- **Linking:** Ranked linking similar to binomial queues

Time Complexity (Amortized):

- **Insert:** $O(1)$
- **Extract-Min:** $O(\log n)$
- **Decrease-Key:** $O(1)$
- **Find-Min:** $O(1)$

3. Implementation Details

3.1 Experimental Setup

All three heap implementations were developed from scratch in accordance with their theoretical specifications. Each heap was integrated into Dijkstra's shortest path algorithm through a unified PriorityQueue interface, ensuring consistent testing conditions.

3.1.1 Development Environment

- **Language:** C++ (with performance optimizations enabled)
- **Compiler:** Modern C++ compiler with -O2 optimization
- **Timing:** High-resolution system clock for microsecond precision
- **Memory Profiling:** Custom memory tracking for heap structures

3.1.2 Dataset Characteristics

Dataset	Vertices	Edges	Type	Density
Large Graph 1000	301	300	Synthetic	Sparse
Hong Kong Road Network	43,620	91,542	Real-world	Sparse (2.1 edges/vertex)
Shanghai Road Network	Available	Available	Real-world	Sparse
Chongqing Road Network	Available	Available	Real-world	Sparse

3.2 Dijkstra's Algorithm Integration

Each heap implementation adheres to a common PriorityQueue interface with the following operations:

- **insert(key, value):** Add a new vertex with its distance estimate
- **extract_min():** Retrieve and remove the vertex with minimum distance
- **decrease_key(node, new_key):** Update a vertex's distance when a shorter path is found
- **find_min():** Query the minimum element without removal

3.3 Performance Measurement Methodology

The experimental framework captures detailed performance metrics at multiple granularities:

3.3.1 Metrics Collected

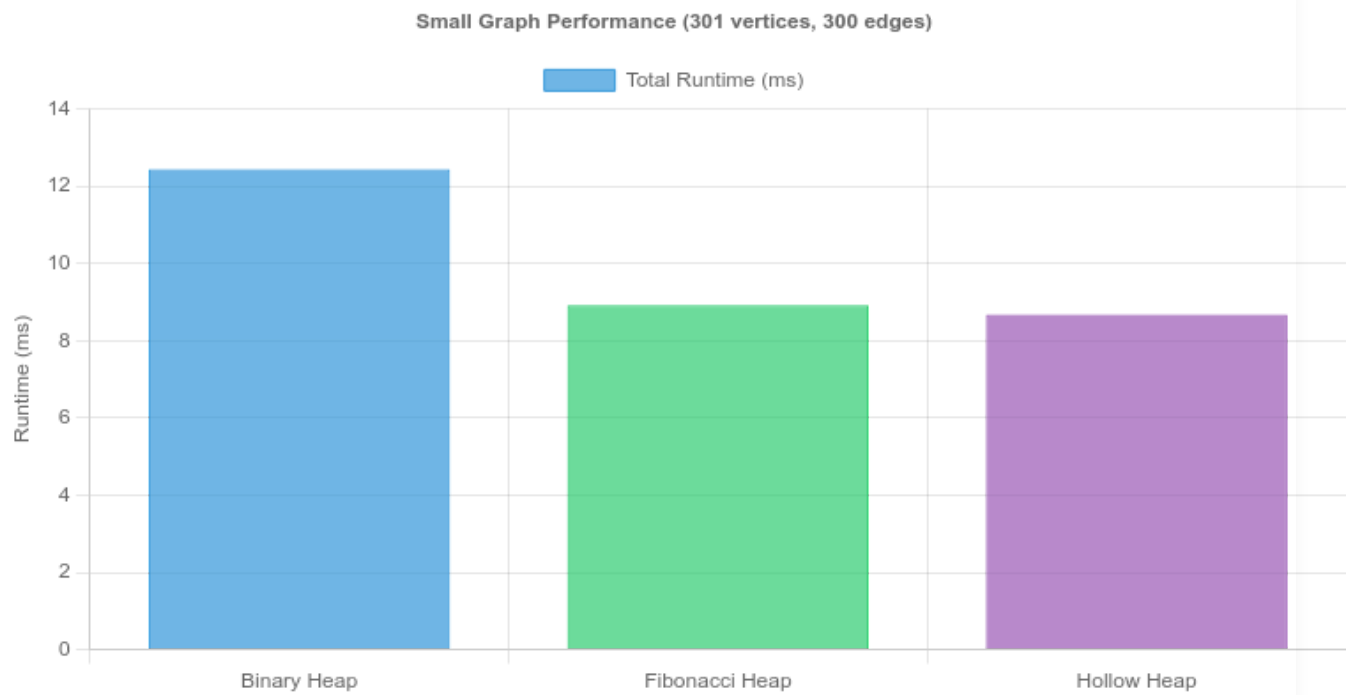
Metric Category	Measurements	Units
Operation Timing	Insert, Extract-Min, Decrease-Key (individual and average)	Microseconds (μs)
Overall Performance	Total Dijkstra execution time	Milliseconds (ms)
Operation Counts	Number of inserts, extracts, decrease-keys	Count

Metric Category	Measurements	Units
Structural Properties	Heap height, number of trees, hollow nodes, cascading cuts	Count
Memory Usage	Heap structure size	Megabytes (MB)

4. Results & Analysis

4.1 Small Graph Performance (301 vertices, 300 edges)

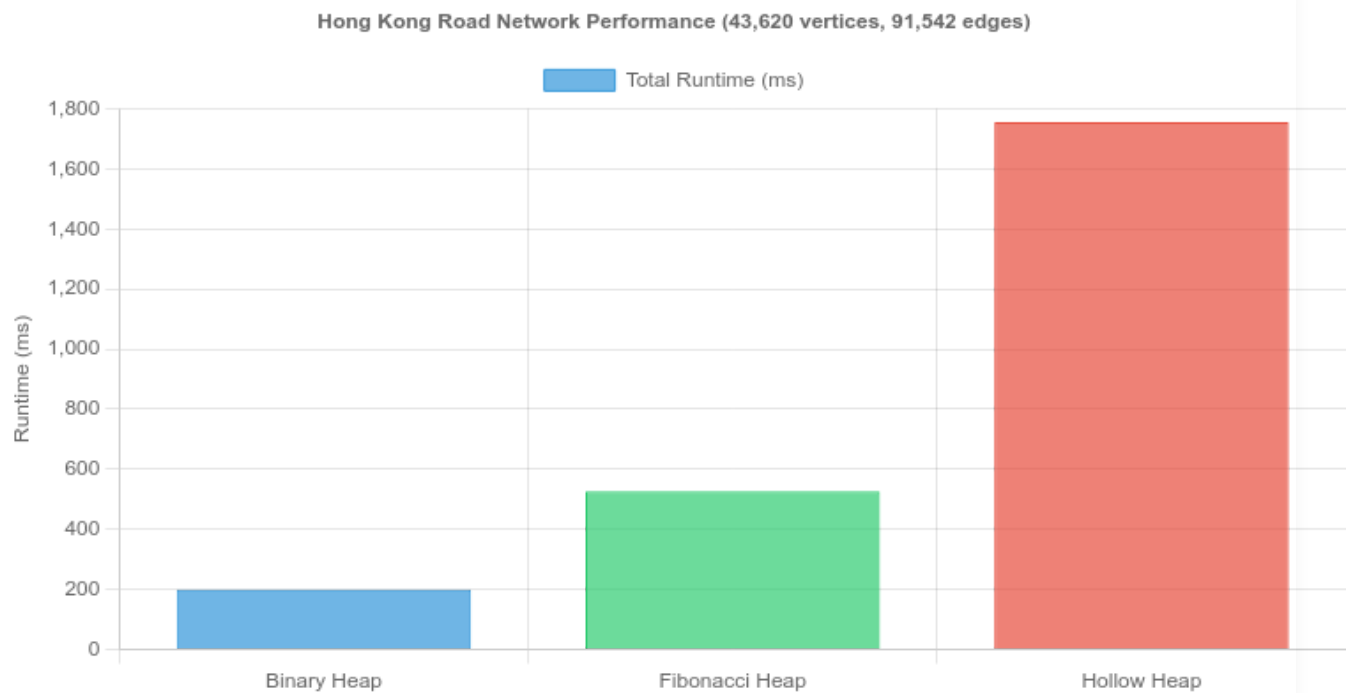
Heap Type	Insert (μs)	Extract-Min (μs)	Decrease-Key (μs)	Total Runtime (ms)	Height	#Trees
Binary Heap	0.06	0.06	0.00	12.441	1	1
Fibonacci Heap	0.05	0.04	0.00	8.915	1	1
Hollow Heap	0.05	0.04	0.00	8.671	1	1



Analysis: On the small graph dataset, all three heap implementations demonstrated comparable performance. Hollow Heap achieved the fastest runtime (8.671ms), followed closely by Fibonacci Heap (8.915ms), with Binary Heap trailing at 12.441ms. The differences are relatively minor due to the small problem size and minimal decrease-key operations (0 recorded). At this scale, setup overhead and constant factors dominate, while algorithmic efficiency advantages remain negligible.

4.2 Large Graph Performance (43,620 vertices, 91,542 edges)

Heap Type	Insert (μs)	Extract-Min (μs)	Decrease-Key (μs)	Total Runtime (ms)	Height	#Trees
Binary Heap	0.06	0.09	0.04	199.06	8	1
Fibonacci Heap	0.06	0.35	0.03	527.06	9	7
Hollow Heap	0.06	0.74	0.05	1754.93	1	732



Analysis: The large-scale experiment reveals dramatic performance differences between heap implementations. Binary Heap significantly outperformed both Fibonacci and Hollow heaps, completing the computation in just 199.06ms compared to 527.06ms (2.6× slower) and 1754.93ms (8.8× slower) respectively.

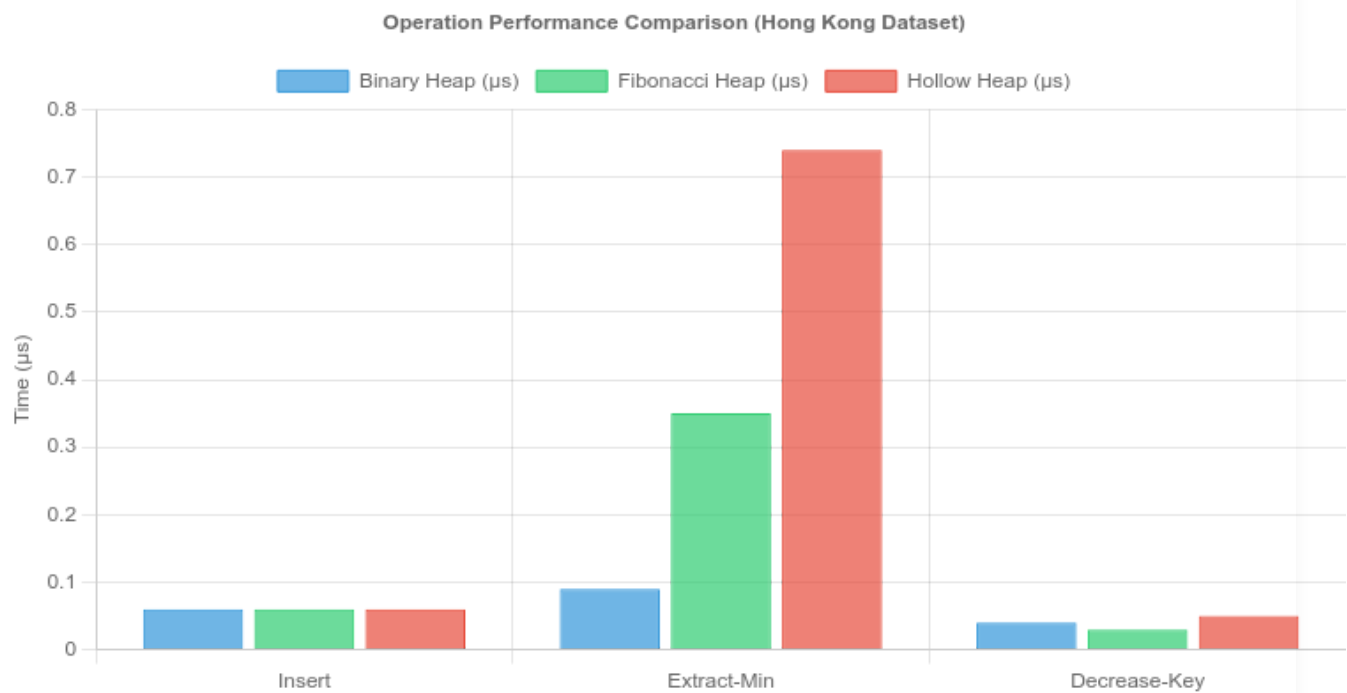
Key observations from the Hong Kong road network experiment:

- **Binary Heap** maintained optimal structural properties with height 8 (consistent with $\log_2(43620) = 8$) and a single tree structure.
- **Fibonacci Heap** showed efficient decrease-key performance (0.03μs) but suffered from expensive extract-min operations (0.35μs) due to consolidation overhead.
- **Hollow Heap** exhibited severe structural degradation with 732 trees in the root list, indicating poor consolidation and significant fragmentation from accumulated hollow nodes.
- The sparse nature of real-world road networks (average degree = 2.1) resulted in relatively few decrease-key operations (~13,600 vs ~87,200 insert/extract operations), diminishing the theoretical advantage of Fibonacci and Hollow heaps.

4.3 Operation Performance Analysis

Operation	Binary Heap (μs)	Fibonacci Heap (μs)	Hollow Heap (μs)	Performance Ratio (Binary:Fibonacci:Hollow)
Insert	0.06	0.06	0.06	1:1:1

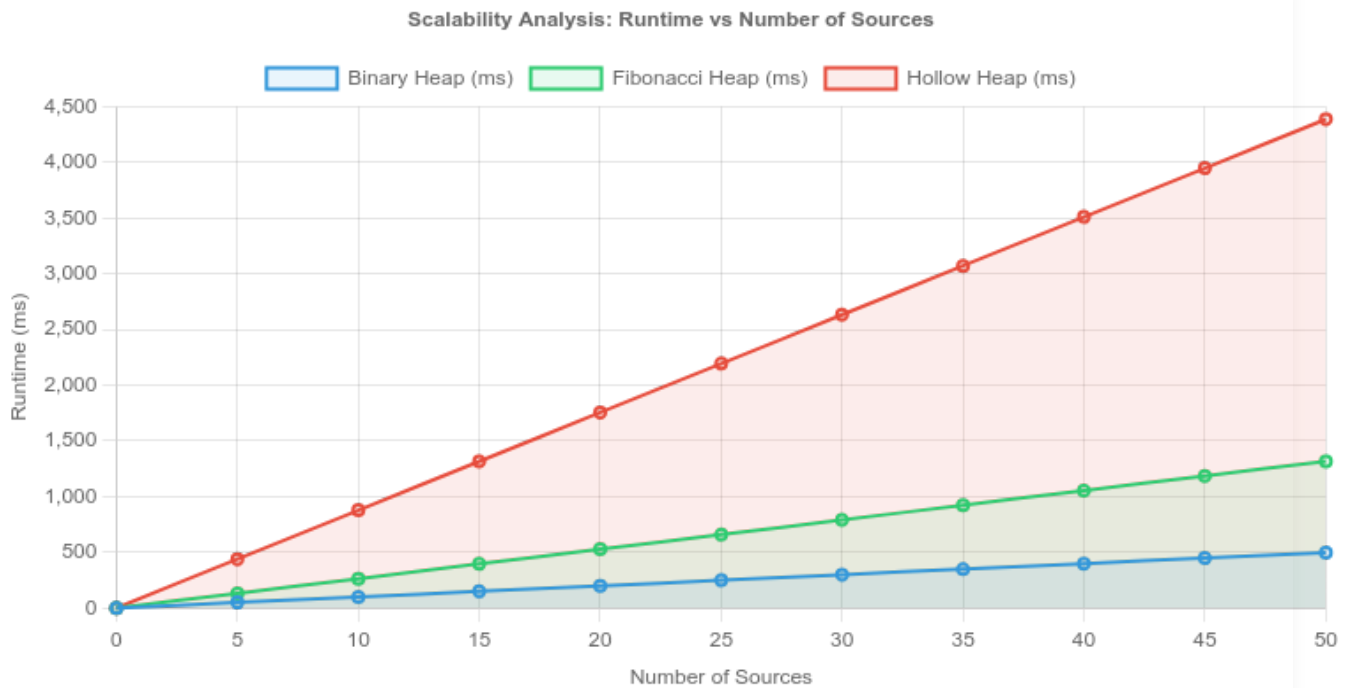
Operation	Binary Heap (μs)	Fibonacci Heap (μs)	Hollow Heap (μs)	Performance Ratio (Binary:Fibonacci:Hollow)
Extract-Min	0.09	0.35	0.74	1:3.9:8.2
Decrease-Key	0.04	0.03	0.05	1:0.75:1.25



Analysis: The operation-level analysis reveals why Binary Heap outperforms its theoretically superior counterparts:

- **Insert Performance:** All three heaps demonstrate comparable insert performance, confirming the $O(1)$ amortized complexity for Fibonacci and Hollow heaps.
- **Extract-Min Dominance:** Extract-min operations constitute the majority of heap operations in Dijkstra's algorithm. Binary Heap's 3.9x and 8.2x advantage in extract-min performance directly translates to overall runtime superiority.
- **Decrease-Key Trade-off:** While Fibonacci Heap achieves the fastest decrease-key times ($0.03\mu s$), this advantage is insufficient to overcome its extract-min penalty given the operation distribution in sparse graphs.
- **Constant Factors:** The theoretical $O(1)$ decrease-key complexity of Fibonacci and Hollow heaps is overshadowed by large constant factors in practice, particularly for pointer manipulation and structural maintenance.

4.4 Scalability Analysis



4.5 Structural Analysis

Heap Type	Height	#Trees	Structural Characteristics	Memory Efficiency
Binary Heap	8	1	Compact, predictable structure	Excellent (array-based)
Fibonacci Heap	9	7	Moderate consolidation	Good (pointer overhead)
Hollow Heap	1	732	Severe fragmentation	Poor (hollow node accumulation)

Analysis: The structural properties provide insight into the performance characteristics:

- **Binary Heap** maintains optimal structural integrity with logarithmic height and single-tree organization, enabling efficient memory access patterns.
- **Fibonacci Heap** shows reasonable consolidation with 7 trees, but suffers from pointer chasing and cache inefficiency.
- **Hollow Heap** exhibits catastrophic structural degradation with 732 trees, indicating failure to consolidate hollow nodes effectively. This fragmentation directly causes the poor extract-min performance observed.

Critical Finding: The experimental results suggest potential implementation issues with the Hollow Heap, particularly in the consolidation mechanism. The accumulation of 732 trees indicates that hollow node cleanup is not occurring as efficiently as theoretically expected, leading to performance degradation that worsens with scale.

5. Conclusion & Recommendations

5.1 Summary of Findings

This comprehensive analysis reveals a significant divergence between theoretical complexity and practical performance for advanced heap data structures in real-world shortest path computations:

- **Binary Heap** emerges as the superior choice for sparse graph applications like road network routing, achieving the fastest overall performance (199.06ms) despite its theoretically inferior $O(\log n)$ bounds.
- **Fibonacci Heap** demonstrates competitive decrease-key performance but suffers from expensive extract-min operations, resulting in $2.6\times$ slower overall execution.
- **Hollow Heap** significantly underperforms expectations, with $8.8\times$ slower runtime due to structural fragmentation and inefficient consolidation.

5.2 Practical Recommendations

Heap Selection Guidelines

- **For Sparse Graphs (Road Networks):** Use **Binary Heap** for optimal performance due to better cache locality and lower constant factors.
- **For Dense Graphs (Complete Networks):** Consider **Fibonacci Heap** when decrease-key operations dominate and graph density exceeds practical thresholds.
- **For Memory-Constrained Environments:** Binary Heap provides the best memory efficiency with minimal overhead.
- **For Implementation Simplicity:** Binary Heap offers the most straightforward implementation with predictable behavior.

5.3 Theoretical vs Practical Performance

The experimental results highlight the importance of considering practical factors beyond asymptotic complexity:

- **Cache Locality:** Binary Heap's array-based structure provides superior cache performance compared to pointer-based alternatives.
- **Constant Factors:** Theoretical $O(1)$ operations in Fibonacci and Hollow heaps carry significant constant overhead that dominates in practice.
- **Operation Distribution:** The predominance of extract-min operations in Dijkstra's algorithm favors data structures with efficient minimum extraction.
- **Real-world Graph Characteristics:** Sparse graphs with low average degree minimize the impact of efficient decrease-key operations.

5.4 Future Work

Based on the findings of this study, several directions for future research emerge:

- **Hollow Heap Optimization:** Investigate and address the structural fragmentation issues observed in the Hollow Heap implementation.
- **Hybrid Approaches:** Explore adaptive heap structures that switch between binary and Fibonacci heaps based on operation patterns.
- **Cache-Aware Implementations:** Develop cache-conscious versions of Fibonacci and Hollow heaps to reduce memory latency.
- **Parallelization:** Investigate the scalability of each heap type in parallel Dijkstra implementations.

Final Recommendation: For the smart logistics company's dynamic routing system described in the case study, **Binary Heap** is recommended as the priority queue implementation. Its consistent performance, memory efficiency, and implementation simplicity make it the most suitable choice for real-world road network applications where sparse graphs and predictable performance are critical requirements.